

Лабораторная работа №1. Среда разработки и первая программа

Дисциплина: Архитектура ЭВМ

Инструменты: .NET SDK 10.0, VS Code + C# Dev Kit, Git Bash

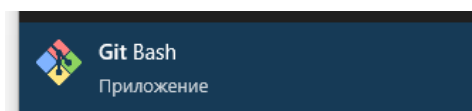
Цели работы:

- Освоить терминал Git Bash как основной инструмент разработчика.
- Создать и запустить первую программу на C# через dotnet CLI.
- Увидеть, **что именно** создаёт компилятор из вашего кода (файлы .dll, .exe, .pdb).
- Получить первичную информацию о компьютере из программы (Environment).
- Завести репозиторий курса на GitHub и сделать первый коммит.

На протяжении всего курса вы будете коммитить каждую работу в **один и тот же** репозиторий **hardware-course_FIO**.

Необходимое программное обеспечение

Нажмите **Win**, введите **Git Bash**, запустите.



Вы увидите:

```
denis@DESKTOP-T3S701S MINGW64 ~  
$
```

Знак **\$** — это приглашение ввести команду. Всё что идёт после него — вы пишете сами. Выполните команду для проверки версии Git:

Выполните следующие команды:

Инструмент	Проверка установки
.NET SDK 10.0+	dotnet --version
VS Code	code --version
Git + Git Bash	git --version

Если хотя бы одна из команд не работает — позовите преподавателя до начала работы.

Ожидаемый вывод (если версии инструментов отличаются, ничего страшного):

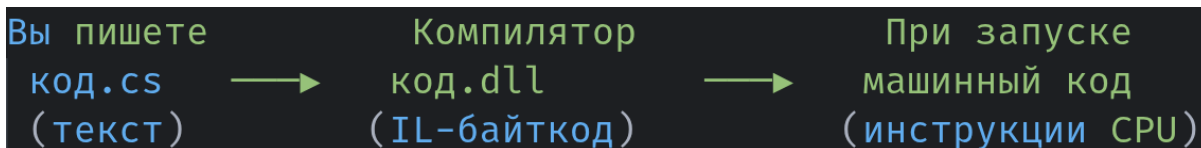
```
denis@DESKTOP-T3S7O1S MINGW64 ~  
$ dotnet --version  
10.0.202  
  
denis@DESKTOP-T3S7O1S MINGW64 ~  
$ code --version  
1.113.0  
cfbea10c5fffb233ea9177d34726e6056e89913dc  
x64  
  
denis@DESKTOP-T3S7O1S MINGW64 ~  
$ git --version  
git version 2.53.0.windows.1
```

Не закрывайте терминал

Теоретическая справка

C# и .NET — что это?

Когда вы пишете код на C#, он **не превращается сразу в команды процессора**. Сначала компилятор переводит текст программы в промежуточный язык **IL** (Intermediate Language). Затем, уже при запуске программы, **JIT-компилятор** (Just-In-Time — «точно в срок»), входящий в состав .NET Runtime, переводит IL в инструкции вашего конкретного процессора.



Благодаря этому одна и та же программа может запускаться на **Windows, Linux и macOS** — достаточно наличия .NET Runtime.

***Сравнение с Python.** В Python каждая строка кода читается и выполняется «на лету» интерпретатором. В C# исходный файл сначала полностью переводится в IL, и только потом запускается. Это даёт скорость: JIT-компилятор успевает оптимизировать код под конкретный процессор. Поэтому C#-программы, как правило, работают быстрее Python-скриптов.*

Мы подробно разберём этот процесс на **Лекции «Путь программы»** и «вскроем» .dll-файлы в **следующих работах**. Сегодня важно увидеть этапы своими глазами.

Терминал — зачем он нужен?

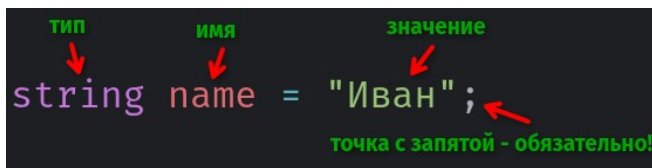
В терминале вы даёте компьютеру команды текстом. Это быстрее и точнее, чем кликать по меню. Все профессиональные разработчики работают в терминале. На Windows мы используем **Git Bash** — он понимает те же команды, что Linux и macOS.

Переменные в C#

В Python:

```
name = "Иван"
```

В С# нужно указать тип:



```
string name = "Иван";
```

Python определяет тип переменной в момент запуска — **динамическая типизация**. С# проверяет типы на этапе компиляции — **статическая типизация**. Если вы попытаетесь записать текст в переменную типа `int`, программа просто не скомпилируется. Это помогает находить ошибки раньше, ещё до запуска.

Часть 1. Терминал

Шаг 1. Открытие Git Bash

Вернитесь в терминал Git Bash.

Шаг 2. Первые команды

Вводите команды по одной, нажимая **Enter** после каждой:

Где я нахожусь?

pwd

Что здесь есть?

ls

Перейти в папку Документы

cd ~/Documents

Символ `~` означает домашнюю директорию текущего пользователя.

Лайфхак: при вводе команд нажимайте **Tab** — терминал сам дополнит команду или путь.

Шаг 3. Создание папки курса

Мы создадим одну папку на весь семестр — все работы будут лежать внутри неё.

mkdir hardware-course_FIO

Где FIO — ваша фамилия (если вас двое пишете две фамилии `_FIO1-FIO2`)

cd hardware-course_FIO

pwd

Ожидаемый результат `pwd`:

```
denis@DESKTOP-T3S701S MINGW64 ~/Documents
$ mkdir hardware-course_Leontev
```

```
denis@DESKTOP-T3S701S MINGW64 ~/Documents
$ cd hardware-course_Leontev/
denis@DESKTOP-T3S701S MINGW64 ~/Documents/hardware-course_Leontev
$ pwd
/c/Users/denis/Documents/hardware-course_Leontev
```

Команда	Что делает
pwd	Print Working Directory — показывает текущую папку
ls	List — список файлов и папок
cd	Change Directory — переход в папку
mkdir	Make Directory — создать папку

Подсказка: стрелка ↑ повторяет предыдущую команду — удобно при опечатках.

Часть 2. GitHub

Git — система контроля версий: она запоминает все изменения в вашем коде.

GitHub — сайт, где хранятся репозитории.

Шаг 4. Обязательный шаг при работе в компьютерных классах

Если вы работаете за общим компьютером, на нём могут остаться учётные данные предыдущего студента. Это приведёт к ошибкам при git push.

Выполняйте следующие шаги в начале каждой пары.

1. Настройте имя и почту Git

git config user.name "ваш_ник_на_github"

git config user.email "ваша_почта_от_github"

Значения должны совпадать с данными вашей учётной записи GitHub!!!

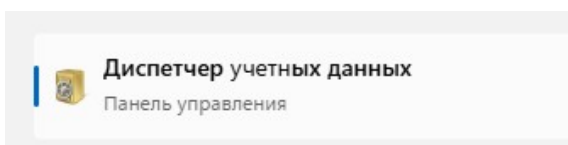
Проверить:

git config user.name

git config user.email

2. Очистите чужие учётные данные Windows

1. Нажмите **Win**, введите «Диспетчер учётных данных».



2. Перейдите во вкладку **Учётные данные Windows**.

Администрирование учетных данных

Просмотр и удаление сохраненных сведений о входе для веб-сайтов, подключенных приложений и сетей.



Учетные данные для Интернета



Учетные данные Windows

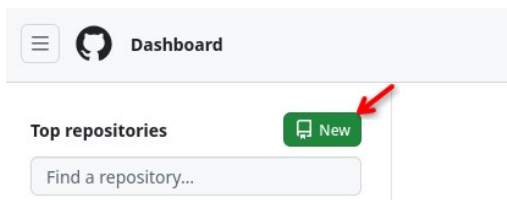
3. Удалите все записи, связанные с github.com.

3. Войдите в GitHub в браузере по умолчанию (обычно Google Chrome). При первом git push откроется окно авторизации — оно подхватит вход из браузера.

Шаг 5. Создание репозитория на GitHub

1. Откройте github.com в браузере по умолчанию (это важно, с него позже будет подтверждение вашего аккаунта).

2. Нажмите **New repository**.



Далее:

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

3. **Repository name:** **hardware-course_FIO**

4. **Visibility:** **Public**

5. Add **README** / **.gitignore** / **license**: ничего не добавлять — мы создадим их вручную.

6. Нажмите **Create repository**.

GitHub покажет инструкции вида:

...or push an existing repository from the command line

```
git remote add origin https://github.com/Paltosik92/hardware-course_Leontev.git
git branch -M main
git push -u origin main
```

Эту ссылку (URL) мы сейчас используем.

Шаг 6. Связывание локальной папки с GitHub

Убедитесь, что вы находитесь в папке **hardware-course_FIO**:

pwd

```
denis@DESKTOP-T3S701S MINGW64 ~/Documents/hardware-course_Leontev
$ pwd
/c/Users/denis/Documents/hardware-course_Leontev
```

Вводим в терминале:

git init -b main

Пояснение:

- `git init` создаёт новый локальный Git-репозиторий;
- параметр `-b main` задаёт имя основной ветки (`main` — современный стандарт вместо устаревшего `master`).

Ожидаемый вывод:

```
denis@DESKTOP-T3S701S MINGW64 ~/Documents/hardware-course_Leontev
$ git init -b main
Initialized empty Git repository in C:/Users/denis/Documents/hardware-course_Leontev/.git/
```

Теперь свяжите локальный репозиторий с удалённым (ссылкой которая была выше в браузере):

git remote add origin <URL_вашего_репозитория>

И после введите в терминале:

git remote -v

Команда `git remote -v` должна показать два адреса — для загрузки (`fetch`) и отправки (`push`).

```
denis@DESKTOP-T3S701S MINGW64 ~/Documents/hardware-course_Leontev (main)
$ git remote -v
origin https://github.com/Paltosik92/hardware-course_Leontev.git (fetch)
origin https://github.com/Paltosik92/hardware-course_Leontev.git (push)
```

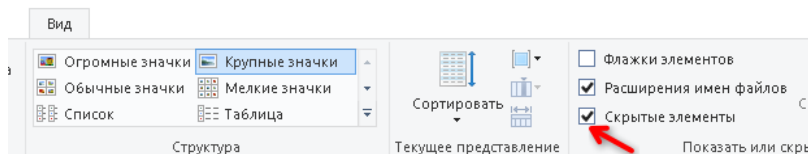
origin — просто короткое имя («псевдоним») для длинного URL. Можно было назвать как угодно, но origin — общепринятое соглашение.

Просмотр структуры Git-репозитория

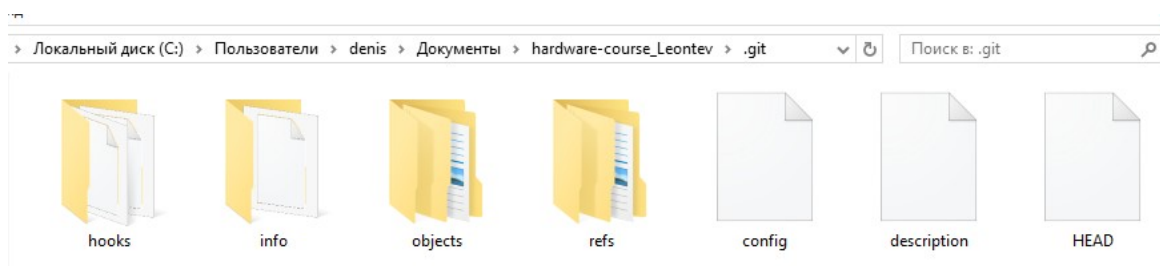
Откройте текущую папку в проводнике:

explorer.

Точка (.) означает «текущая директория». Если скрытая папка .git не видна — в проводнике: **Вид → Скрытые элементы**.



Откройте скрытую папку .git.



- objects/ — здесь хранятся все версии всех файлов в сжатом виде;
- refs/ — указатели на ветки (например, на последний коммит ветки main);
- HEAD — текстовый файл с именем текущей ветки;
- config — настройки этого репозитория (в том числе адрес origin).

Часть 3. Первая программа

Шаг 7. Создание проекта Lab01

Находясь в папке hardware-course_FIO, выполните:

dotnet new console -n Lab01

Ожидаемый вывод:

```
denis@DESKTOP-T3S701S MINGW64 ~/Documents/hardware-course_Leontev (main)
$ dotnet new console -n Lab01
Шаблон "Консольное приложение" успешно создан.

Идет обработка действий после создания...
Восстановление C:\Users\denis\Documents\hardware-course_Leontev\Lab01\Lab01.csproj:
Восстановление выполнено.
```

Далее в терминале:

cd Lab01

ls

Пояснение команды:

- **dotnet new console** — создаёт шаблон консольного приложения;
- **-n Lab01** — имя проекта (и папки).

Ожидаемая структура:

```
hardware-course/      ← корень репозитория (здесь делали git init)
├── Lab01/             ← проект первой лабораторной
│   ├── Lab01.csproj   ← настройки проекта (XML)
│   ├── Program.cs     ← ваш код
│   └── obj/           ← временные файлы сборки (появятся после restore)
```

Файл **.csproj** — это **XML-файл**, описывающий конфигурацию проекта: версию .NET, тип выходного файла, зависимости.

Шаг 8. Открытие в VS Code

Выполните в терминале:

code .

Откроется VS Code с открытой папкой **Lab01**. В боковой панели видна структура проекта.

Если VS Code предложит установить расширение C# Dev Kit — соглашайтесь. Без него не будет подсветки синтаксиса и автодополнения.

Откройте **встроенный терминал** сочетанием **Ctrl + `** или **Ctrl + J**.

Все команды ниже выполняются в этом терминале.

Шаг 9. Создание файла .gitignore

В **корневой** папке репозитория (hardware-course_FIO, не в Lab01) создадим .gitignore. Он укажет Git, какие файлы не отслеживать.

cd .. # поднялись в hardware-course_FIO

touch .gitignore

code .gitignore

Мы открыли **.gitignore** в VS Code, в него добавьте:

Скомпилированные файлы и сборка

bin/

obj/

Локальные настройки VS Code

.vscode/

Системный мусор

.DS_Store

Thumbs.db

Почему это важно:

- **bin/** и **obj/** — папки, которые .NET создаёт **автоматически** при каждой сборке. Класть их в репозиторий бессмысленно — их размер может быть в десятки раз больше исходного кода.
- **.gitignore** нужно создавать **до первого git add**. Если файл уже попал в индекс, .gitignore его не остановит.

Шаг 10. Первый коммит

Представьте посылку: *git add* — положить вещи в коробку; *git commit* — заклеить коробку и написать адрес. Только после commit Git запоминает это состояние навсегда.

1. Проверьте состояние репозитория в терминале:

git status

Git покажет список неотслеживаемых файлов (Untracked files).

```
denis@DESKTOP-T3S701S MSYS ~/Documents/hardware-course_Leontev (main)
• $ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore
    Lab01/

nothing added to commit but untracked files present (use "git add" to track)
```

2. Добавьте все файлы в индекс:

git add .

Точка (.) — «всё в текущей папке, кроме игнорируемого».

3. Снова проверьте состояние:

git status

```
denis@DESKTOP-T3S701S MSYS ~/Documents/hardware-course_Leontev (main)
• $ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   .gitignore
    new file:   Lab01/Lab01.csproj
    new file:   Lab01/Lab01.sln
    new file:   Lab01/Program.cs
```

Теперь файлы в разделе Changes to be committed (подсвечены зелёным).

4. Создайте коммит:

git commit -m "Lab01: initial project setup"

```
denis@DESKTOP-T3S701S MSYS ~/Documents/hardware-course_Leontev/Lab01 (main)
$ git commit -m "Lab01: initial project setup"
[main (root-commit) ab9ae9f] Lab01: initial project setup
4 files changed, 43 insertions(+)
create mode 100644 .gitignore
create mode 100644 Lab01/Lab01.csproj
create mode 100644 Lab01/Lab01.sln
create mode 100644 Lab01/Program.cs
```

В ответе Git покажет:

- [main (root-commit) <хеш>] — первый («корневой») коммит в ветке main;
- <хеш> — первые 7 символов уникального идентификатора длиной 40 СИМВОЛОВ;
- количество изменённых файлов.

5. Посмотрите историю:

git log

```
denis@DESKTOP-T3S701S MSYS ~/Documents/hardware-course_Leontev/Lab01 (main)
$ git log
commit ab9ae9f3549cfb3301f6de1836258701d8c2b13f (HEAD -> main)
Author: Paltosik92 <denis.leontev92@gmail.com>
Date: Mon Apr 27 13:24:01 2026 +0300

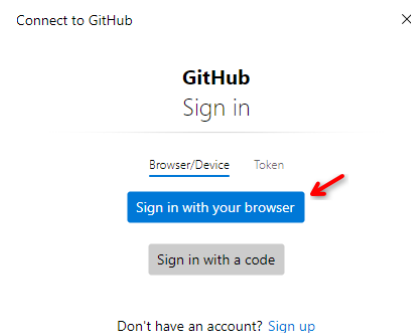
    Lab01: initial project setup
```

Строка (HEAD -> main) означает: указатель HEAD (где вы сейчас) находится на ветке main.

6. Отправьте на GitHub:

git push -u origin main

При первом push откроется окно авторизации — войдите через браузер.



После успеха обновите страницу репозитория на GitHub — там появится папка Lab01.

main

Go to file

+

<> Code

Paltosik92 Lab01: initial project setup

ab9ae9f · 2 minutes ago

Lab01	Lab01: initial project setup	2 minutes ago
.gitignore	Lab01: initial project setup	2 minutes ago

Вывод в терминале:

```
denis@DESKTOP-T3S701S MSYS ~/Documents/hardware-course_Leontev/Lab01 (main)
$ git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 1.13 KiB | 1.13 MiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Paltosik92/hardware-course_Leontev.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Говорит о том что произошла успешная отправка.

Часть 4. Что создал компилятор?

Это ключевой архитектурный момент первой работы. Мы сейчас увидим глазами, во что превращается ваш .cs-файл.

Шаг 11. Запустим программу

Вернитесь в папку проекта и выполните:

cd Lab01

dotnet run

Ожидаемый результат:

```
Hello, World!
```

Теперь посмотрим, что **dotnet** положил на диск:

ls -la

```
denis@DESKTOP-T3S701S MSYS ~/Documents/hardware-course_Leontev/Lab01 (main)
$ ls -la
total 10
drwxr-xr-x 1 denis 197121  0 Apr 27 13:18 ./
drwxr-xr-x 1 denis 197121  0 Apr 27 13:19 ../
drwxr-xr-x 1 denis 197121  0 Apr 27 13:18 bin/
-rw-r--r-- 1 denis 197121 253 Apr 27 13:14 Lab01.csproj
-rw-r--r-- 1 denis 197121 1104 Apr 27 13:18 Lab01.sln
drwxr-xr-x 1 denis 197121  0 Apr 27 13:18 obj/
-rw-r--r-- 1 denis 197121  40 Apr 27 13:14 Program.cs
```

Обратите внимание на новые папки **bin/** и **obj/** — раньше их не было.

Шаг 12. Исследуем папку bin/

ls bin/Debug/net10.0/

Вы увидите примерно такой список:

```
denis@DESKTOP-T3S701S MSYS ~/Documents/hardware-course_Leontev/Lab01 (main)
$ ls bin/Debug/net10.0/
Lab01.deps.json Lab01.dll* Lab01.exe* Lab01.pdb Lab01.runtimeconfig.json
```

Что это за файлы:

Файл	Что это
Lab01.dll	Ваша программа в виде IL-байткода (то самое промежуточное представление). Именно этот файл запускает .NET Runtime.
Lab01.exe	Небольшой исполняемый «запускатель» под Windows, который находит Runtime и передаёт ему .dll.
Lab01.pdb	Символы отладки (Program DataBase). Нужны для пошаговой отладки и красивых сообщений об ошибках.
Lab01.deps.json	Список зависимостей (каких DLL требует программа).
Lab01.runtimeconfig.json	На какой версии .NET запускать программу.

Посмотрите размер:

ls -lh bin/Debug/net10.0/

Ваш Program.cs — меньше килобайта. А Lab01.dll — несколько килобайт. Почему? Потому что компилятор **упаковал** туда IL-инструкции, метаданные, строки, ссылки на типы. На лекции мы разберём, как устроен этот файл изнутри.

Шаг 13. Запуск скомпилированного файла напрямую

dotnet run делает две вещи сразу: собирает проект и запускает. Сделаем то же самое по шагам:

только сборка

dotnet build

только запуск уже собранного

dotnet bin/Debug/net10.0/Lab01.dll

```
denis@DESKTOP-T3S701S MSYS ~/Documents/hardware-course_Leontev/Lab01 (main)
$ dotnet build
Восстановление завершено (0,6 с)
  Lab01 net10.0 успешно выполнено (0,2 с) → bin\Debug\net10.0\Lab01.dll

Сборка успешно выполнено через 1,0 с

denis@DESKTOP-T3S701S MSYS ~/Documents/hardware-course_Leontev/Lab01 (main)
$ dotnet bin/Debug/net10.0/Lab01.dll
Hello, World!
```

Результат будет одинаковый. Главный вывод: **запускается именно .dll**, а не .cs. Исходный текст программы компьютеру уже не нужен.

Шаг 14. Очистка сборки

Иногда нужно удалить результаты сборки (например, чтобы пересобрать с нуля):

dotnet clean

ls bin/Debug/net10.0/

Папка будет пуста или удалена

Часть 5. Визитка системы

Теперь соберём маленькую программу, которая расскажет о **компьютере**, на котором она запускается. Используем класс `Environment` — он даёт доступ к информации об ОС и железе прямо из кода.

Шаг 15. Новый код Program.cs

Откройте **Lab01/Program.cs** в VS Code и замените всё содержимое на (весь код переписываем вручную):

Визитка системы

```
Console.ForegroundColor = ConsoleColor.Cyan;
Console.WriteLine("ВИЗИТКА СИСТЕМЫ");
Console.ResetColor();
```

Данные студента

```
string studentName = "Ваше Имя"; // ← замените на своё
string studentGroup = "ИСП-221"; // ← замените на свою группу
Console.ForegroundColor = ConsoleColor.Yellow;
Console.WriteLine("\n[Студент]");
Console.ResetColor();
Console.WriteLine($"Имя:      {studentName}");
Console.WriteLine($"Группа:   {studentGroup}");
Console.WriteLine($"Дата:     {DateTime.Now:dd.MM.yyyy HH:mm}");
```

Компьютер

```
Console.ForegroundColor = ConsoleColor.Yellow;
Console.WriteLine("\n[Компьютер]");
Console.ResetColor();
Console.WriteLine($"Имя машины:    {Environment.MachineName}");
Console.WriteLine($"Пользователь:  {Environment.UserName}");
Console.WriteLine($"ОС:           {Environment.OSVersion}");
Console.WriteLine($"64-битная ОС: {Environment.Is64BitOperatingSystem}");
Console.WriteLine($"Версия .NET:   {Environment.Version}");
```

Процессор

```
Console.ForegroundColor = ConsoleColor.Yellow;
Console.WriteLine("\n[Процессор]");
Console.ResetColor();
Console.WriteLine($"Логических ядер: {Environment.ProcessorCount}");
```

Память

```
Console.ForegroundColor = ConsoleColor.Yellow;
Console.WriteLine("\n[Память]");
Console.ResetColor();
long workingSetMb = Environment.WorkingSet / 1024 / 1024;
Console.WriteLine($"Память процесса: {workingSetMb} МБ");
Console.WriteLine($"Размер указателя: {IntPtr.Size * 8} бит");
```

Итог

```
Console.ForegroundColor = ConsoleColor.Green;
Console.WriteLine("\nПрограмма выполнена успешно!");
Console.ResetColor();
```

Пояснения к коду

Типы данных (знакомимся на интуитивном уровне — подробно будет в ПР2):

- **string** — текст;
- **int** — целое число;
- **long** — большое целое число;
- **bool** — да / нет (true / false).

Интерполяция строк `$"..."` — позволяет подставлять значения переменных прямо в строку через `{ }`. Это удобнее, чем склеивать через `+`.

Console.WriteLine выводит строку и переходит на новую. **Console.Write** — без перевода строки.

ConsoleColor — это **перечисление (enum)**: заранее заданный список именованных значений. Подробно изучим во 2 семестре. Важная особенность:

Цвет консоли — **глобальная настройка процесса**. Если не вернуть его через `Console.ResetColor()`, то и все последующие сообщения (включая ошибки .NET) будут в выбранном цвете. В худшем случае красные системные ошибки «солятся» с красным текстом программы.

Environment — специальный класс .NET, который знает об окружении программы:

- **MachineName** — имя компьютера;
- **OSVersion** — версия операционной системы;
- **ProcessorCount** — количество **логических** ядер (с учётом Hyper-Threading);
- **WorkingSet** — сколько памяти занимает **ваш процесс** прямо сейчас (в байтах);
- **Is64BitOperatingSystem** — 64-битная ли ОС.

`IntPtr.Size * 8` — размер указателя в битах. На 64-битной системе это 64, на 32-битной — 32. Почему это важно для железа — узнаем на лекции про «Процессор».

Шаг 16. Запуск

Сохраните файл (**Ctrl+S**) и в терминале:

dotnet run

Ожидаемый результат (значения у вас свои):

```
$ dotnet run
ВИЗИТКА СИСТЕМЫ

[Студент]
Имя: Ваше Имя
Группа: ИСП-221
Дата: 09.06.2026 12:44

[Компьютер]
Имя машины: DENIS
Пользователь: leontev
ОС: Microsoft Windows NT 10.0.26200.0
64-битная ОС: True
Версия .NET: 10.0.8

[Память]
Память процессора: 21 Мб
Размер указателя: 64 бит

Программа выполнена успешно!
```

Сравните свой результат с соседом! У соседних ПК могут отличаться:

- имя машины,
- размер занятой памяти.

Зафиксируем:

cd ..

git add .

git commit -m "Lab01: system business card"

git push

Проверьте на GitHub — обновлённый Program.cs должен быть виден в браузере.

Самостоятельные задания

Задание 1. Своя визитка ★

Измените блок [Студент] так, чтобы он выводил:

- ваше полное имя;
- группу;

- любимый язык программирования;
- сколько лет вы программируете.

Выберите **свою** цветовую схему (не обязательно как в примере). Все доступные цвета можно посмотреть, начав писать `ConsoleColor`. — VS Code подскажет через автодополнение.

Задание 2. Все цвета консоли ★★

Напишите фрагмент кода, который выводит **все 16 цветов** `ConsoleColor` — каждый своим цветом. Должно получиться что-то вроде:

Black ← чёрный текст

DarkBlue ← тёмно-синий текст

...

White ← белый текст

Полный список значений `ConsoleColor`:

`Black`, `DarkBlue`, `DarkGreen`, `DarkCyan`, `DarkRed`, `DarkMagenta`, `DarkYellow`, `Gray`, `DarkGray`, `Blue`, `Green`, `Cyan`, `Red`, `Magenta`, `Yellow`, `White`

Не забудьте в конце вызвать `Console.ResetColor()`.

Закоммитьте:

git add .

git commit -m "Lab01 task 3: all console colors"

git push

Контрольные вопросы

Создайте в корне репозитория (**hardware-course_FIO/**, не внутри **Lab01/**) файл **README.md** и ответьте на вопросы:

1. Чем `dotnet run` отличается от `dotnet build`? Какая команда что делает?
2. Какие файлы появляются в `bin/Debug/net10.0/` после сборки? Что в них лежит? Зачем нужен `.pdb`?
3. Что делает компилятор с вашим `.cs`-файлом перед запуском? Почему процессор не может выполнить текст исходного кода напрямую?
4. Зачем нужен `.gitignore`? Что произойдёт, если положить в репозиторий папку `bin/`?
5. Какой тип данных (`int`, `string`, `double`, `bool`) вы бы выбрали для хранения: фамилии; возраста; среднего балла; признака «студент отчислен / не отчислен»?

6. Что выведет Environment.ProcessorCount на вашем компьютере? А на ноутбуке соседа? Почему цифры могут отличаться?

7. Зачем вызывать Console.ResetColor() после смены цвета?

Закоммитьте:

git add .

git commit -m "Lab01: README with control questions"

git push

Чеклист сдачи

Перед тем, как закрыть VS Code, проверьте:

- На GitHub в репозитории hardware-course_FIO видна папка Lab01/.
- В Lab01/ лежат Program.cs и Lab01.csproj.
- В Lab01/ **НЕТ** папок bin/ и obj/ (их отрезал .gitignore).
- В корне репозитория есть README.md с ответами на контрольные вопросы.
- git log показывает минимум 5 коммитов.
- Программа запускается командой dotnet run и выводит вашу визитку.

Вышлите репозиторий преподавателю на образовательном портале.

Не забудьте подписать ваши ФИО, название дисциплины и номер работы.

Помощь при проблемах

Удаление папки .git

Внимание: Делать это нужно очень осторожно, так как при удалении потеряются все настройки и коммиты!

Вы можете удалить папку **.git** (которая содержит всю историю проекта, конфигурационные файлы, объекты, ветки и метаданные, необходимые для работы системы контроля версий), после чего заново выполнить все команды:

rm -rf .git



Частая проблема студентов: Папка .git создавалась не в том месте (например, в корневой директории документов), из-за чего проект не мог нормально запуститься.

Совет на будущее: Если столкнулись с нестандартным поведением Git, визуально проверьте, нет ли папки **.git** в вышележащих папках.